

AI TRAINING – DAY 3 ASSIGNMENT

Python Operators & Strings: Manipulate → Extract → Clean → Format

Practical + MCQ + Debugging + String Utility Project

1. Learning Goals

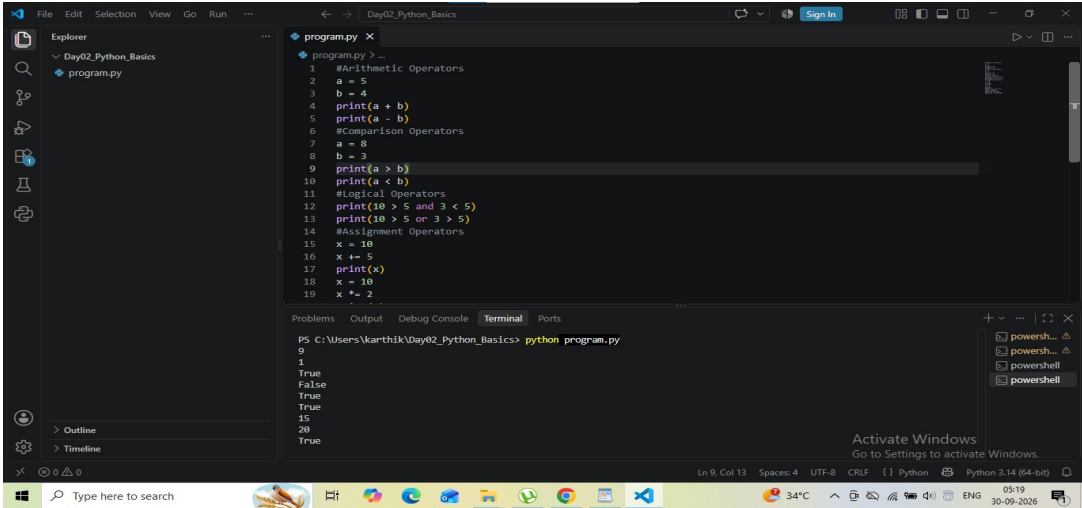
- Understand arithmetic, comparison, logical, assignment and membership operators.
- Understand strings, indexing, slicing and common string methods.
- Use f-strings to create readable dynamic output.
- Identify common operator and string errors.
- Combine all topics to build a practical text utility.
- Explain your solution in your own words.

2. Assignment Rules

- Type the programs yourself. Do not copy complete solutions.
- Before running code, predict the output.
- Test strings with spaces, uppercase/lowercase, empty input and unexpected input.
- You may use AI to explain a concept or error after attempting it yourself.
- For every practical problem write Input → Processing → Output.
- Keep at least one error you faced and explain how you fixed it.

3. Part A – Operators Exploration

Create operators.py and experiment with each group. Write at least 2 examples for every group.



```
1 #Arithmetic Operators
2 a = 5
3 b = 4
4 print(a + b)
5 print(a - b)
6 #Comparison Operators
7 a = 8
8 b = 3
9 print(a > b)
10 print(a < b)
11 #Logical Operators
12 print(10 > 5 and 3 < 5)
13 print(10 > 5 or 3 > 5)
14 #Assignment Operators
15 x = 10
16 x += 5
17 print(x)
18 x = 10
19 x *= 2
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\karthik\Day02_Python_Basics> python program.py
9
1
True
False
True
True
15
20
True
```

Activate Windows
Go to Settings to activate Windows.

Operator group	Operators / your observation
Arithmetic	+ - * / // % ** Observation: it is used to do mathematical calculations Example: a=5 b=4 print(a+b)# addition print(a-b)# sub output: 9 1
Comparison	== != > < >= <= Observation: it is used to compare two values Example: a=8 b=3 print(a > b) print(a < b) output: True False
Logical	and or not Observation: it is used to combine or change conditions Example: print(10 > 5 and 3 < 5) print(10 > 5 or 3 > 5) output: True True
Assignment	= += -= *= /= Observation: it is used to assign or update values Example: x = 10 x += 5 print(x) x = 10 x *= 2 print(x) output: 15 20
Membership	in / not in Observation: it is used to check if a value is present or not Example: name = "Lasya" print("L" in name) print("O" not in name) output: True True

4. Part B – Operator Thinking

1. What is the difference between / and //?

/ it indicate the division and it is used to divide one number to another number

// it indicate the floor division it use to get the whole number result of a division

2. When would % be useful in a real application?

We can use % to check whether a number is even or odd.

3. What does ** do?

** is used to calculate the power of a number

4. What is the difference between = and ==?

= It indicate the equal and it is used to assign a value to a variable

== it indicate the double equal and it is used to compare two values and check whether they are equal

5. What is the difference between != are not?

!=is used to check whether two values are different.

not is a condition which is used to change the existing value to it oppsite value

6. What is the result of 10 > 5 and 3 < 2? Explain.

The result is False because 10 > 5 is True, but 3 < 2 is False. The and operator returns True only when both conditions are True

7. What is the result of 10 > 5 or 3 < 2? Explain.

The result is True because 10 > 5 is True.but 3<2 is False but

The or operator returns True if at least one condition is True

8. What does not True produce?

The not operator changes True to False and False to True

9. What does 'Python' in 'I am learning Python' return?

The in operator checks whether a value is present in a string. Since "Python" is present in "I am learning Python", it returns True.

10. When would not in be useful?

not in is useful when we want to check whether a value is not present in a string, list, or other collection.

5. Part C – String Basics Lab

Use your own name or another sentence and run these experiments.

1. Create a string and print it.

```
name = "Lasya"  
print(name)
```

output:Lasya

2. Print its length using len().

len() is used to find the number of characters in a string

```
name="lasya"
```

```
print(len(name))  
output: 5
```

3. Print the first character using [0].

```
name = "collage"  
print(name[0])  
output: c
```

4. Print the last character using [-1].

```
name="collage"  
print(name[-1])  
output:e
```

5. Print the first 3 characters using slicing.

```
name="collage"  
print(name[0:3])  
output: c o l
```

6. Print the last 3 characters using slicing.

```
name="collage"  
print(name[-3:])  
output: a g e
```

7. Print every second character.

```
name = "collage"  
print(name[::2])  
output:c l a e
```

8. Reverse the string using slicing.

```
name = "collage"  
print(name[::-1])  
output: e g a l l o c
```

9. Try an invalid index. What error occurs?

```
name = "collage"  
print(name[10])  
output:index error
```

10. Try changing one character directly. What happens and why?

```
name = "collage"  
name[0] = "C"  
output : error
```

become you cannot change a character directly in a string because strings are immutable

6. Part D – String Methods Lab

Experiment with these methods and explain what each one does.

Method	Your example	What does it do?
lower()	"LASYA".lower()	Converts capital letters to lowercase
upper()	"lasya".upper()	Converts lowercase letters to uppercase
title()	"introduction".title()	Converts the first letter of

		each word to uppercase
strip()	" hello .,strip()	Removes spaces from the beginning and end
replace()	"i like html".replace("html", "python")	Replaces one part of a string with another.
split()	"I am Lasya".split()	Divides a string into separate words or items
join()	".join(["I", "am", "Lasya"])	Combines separate items into one string.
startswith()	"Python".startswith("Py")	Checks whether the string starts with a specific value
endswith()	"python".endswith("on")	Checks whether a string ends with a specific word or letters.
find()	"I like Python".find("Python")	Finds where a word is in the string
count()	"collage".count("a")	Counts how many times something appears.

7. Part E – Indexing & Slicing Challenge

Use: text = "Artificial Intelligence"

1. Print the first character.
print(text[0]) output:A
2. Print the last character.
print(text[-1]) output:e
3. Print the first 10 characters.
print(text[:10]) output:Artificial
4. Extract the word Intelligence.
print(text[11:]) output:Intelligence
5. Reverse the complete string.
print(text[::-1]) output:ecnegilletnl laicifitrA
6. Print every second character.
print(text[::2]) output: A t f c a _ n e l i e c
7. Print characters from index 3 to 12.
print(text[3:13]) output:ificial ln
8. Find the index of the first 'i'.
print(text.find("i")) output:3
9. Count the number of 'i' characters.
print(text.count("i")) output:4
10. Predict what text[100] will do, then run it.
print(text[100]) output: error

8. Part F – f-Strings

1. Create name, age and city variables and display them in one sentence using an f-string.

```

name = "Lasya"
age = 20
city = "Nellore"
print(f"My name is {name}, I am {age} years old, and I live in {city}.")
output:My name is Lasya, I am 20 years old, and I live in Nellore.

```

2. Create a product, price and quantity and display a formatted bill.

```

product = "Pen"
price = 20
quantity = 3
total = price * quantity
print(f"Product: {product}")
print(f"Price: ₹{price}")
print(f"Quantity: {quantity}")
print(f"Total: ₹{total}")
output:Product: Pen
        Price: ₹20
        Quantity: 3
        Total: ₹60

```

3. Display student name, total marks and average.

```

student_name = "Lasya"
total_marks = 450
average = 90
print(f"Student Name: {student_name}")
print(f"Total Marks: {total_marks}")
print(f"Average: {average}")
output:Student Name: Lasya
        Total Marks: 450
        Average: 90

```

4. Use an f-string to display a calculation result.

```

a = 15
b = 5
result = a + b
print(f"{a} + {b} = {result}")
output:15+5=20

```

5. Research decimal formatting and display a price with exactly 2 decimal places.

```

price = 20.2
print(f"Price: {price:.2f}")
output:20.20

```

9. Part G – MCQs: Basic

1. Which operator means multiplication?

- x
- B. *✓
- C. %
- D. //

2. What does % return?

- Division

- B. Quotient
- C. Remainder✓
- D. Power

3. What is the first string index?

- 0✓
- B. 1
- C. -1
- D. first

4. What is len('Python')?

- 5
- B. 6✓
- C. 7
- D. Error

5. Which gets the last character?

- text[0]
- B. text[-1]✓
- C. text[last]
- D. text[1]

6. Which converts text to lowercase?

- lower()✓
- B. small()
- C. down()
- D. lowercase

7. Which removes leading/trailing spaces?

- trim()
- B. clean()
- C. strip()✓
- D. remove()

8. Which checks membership?

- inside

- B. in✓
- C. has
- D. contains

9. Which is an f-string?

- f"Hello {name}"✓
- B. "f Hello"
- C. format f(name)
- D. string(name)

10. Which compares equality?

- =
- B. ==✓
- C. ===
- D. equals

10. Part H – MCQs: Medium

1. What is 17 // 5?

- 3✓
- B. 3.4
- C. 2
- D. 4

2. What is 17 % 5?

- 3
- B. 2✓
- C. 3.4
- D. 0

3. What does 'Python'[1:4] return?

- Pyt
- B. yth✓
- C. ytho
- D. tho

4. What does 'Python'[-3:] return?

- tho
- B. hon✓
- C. yth
- D. Python

5. What does ' hello '.strip() return?

- ' hello'
- B. 'hello '
- C. 'hello✓'
- D. Error

6. What does 'hello'.upper() return?

- hello
- B. HELLO✓
- C. Hello
- D. Error

7. What does 'a,b,c'.split(',') return?

- abc
- B. ['a','b','c']✓
- C. ('a','b','c')
- D. Error

8. Which checks if AI occurs in text?

- text.contains('AI')
- B. 'AI' in text✓
- C. text.has('AI')
- D. contains(text,'AI')

9. What happens with text[50] when text has 10 characters?

- None
- B. "
- C. IndexError✓
- D. False

10. Why use f-strings?

- Insert values/expressions into strings cleanly✓
- B. Make strings immutable
- C. Convert values to int
- D. Remove spaces

11. Part I – MCQs: Thinking

1. Which is true about strings?

- They can always be changed character-by-character
- B. They are immutable✓
- C. They only contain letters
- D. They cannot be sliced

2. Best way to clean ' PHANI gorantla ' for display?

- strip().title()
- B. title().strip()
- C. Both A and B✓
- D. clean()

3. What does text[::-1] do?

- Deletes text
- B. Reverses text✓
- C. Sorts text
- D. Gets first character

4. Best simple test for '@' in an email?

- '@' in email✓
- B. email == '@'
- C. email.contains('@')
- D. email.has('@')

5. replace() generally:

- Returns a transformed string✓
- B. Always changes the original string in place
- C. Converts to list
- D. Deletes the variable

6. `x='5', y=2`. Which gives numeric 10?

- `x*2`
- B. `int(x)*2`✓
- C. `str(x)*2`
- D. `x+2`

7. Which formats 12.5 as 12.50?

- `f'{price:.2f}'`✓
- B. `f'{price:2}'`
- C. `f'{price.float}'`
- D. `f'{price.decimal(2)}'`

8. What should usually happen before validating text entered with spaces?

- `strip()`✓
- B. `reverse()`
- C. `split()` always
- D. replace with None

9. A common cause of `IndexError` is:

- Invalid index✓
- B. Using an f-string
- C. `lower()`
- D. +

10. Why keep raw input and cleaned input conceptually separate?

- It makes transformations easier to test✓
- B. Python requires it
- C. Strings cannot be stored
- D. It prevents every error

12. Part J – 5 Practice Problems

Problem	Task	Topics
Practice 1 – Username Cleaner	Ask for a username, remove surrounding spaces, convert to lowercase and display the cleaned value.	strip + lower + f-string
Practice 2 – Name Formatter	Ask first and last names in	strip + title + f-string

	random case and display a properly formatted full name.	
Practice 3 – Word Analyzer	Ask for a sentence. Display length, first character, last character, spaces count and vowel count. Do not use loops.	len + indexing + count
Practice 4 – Email Inspector	Ask for an email. Check '@', check whether it ends with '.com' or '.in', and show the position of '@'.	in + endswith + find
Practice 5 – Product Code Extractor	For code IND-MOB-2026-001, extract country, category, year and serial using slicing.	indexing + slicing + f-string

For each: Input → Expected Output → Code → 2 test cases → 1 edge case → explanation.

Practice 1 – Username Cleaner

Input: Lasya

Expected Output: lasya

Code:

```
username = input("Username: ")
clean_user = username.strip().lower()
print(f"Cleaned Username: {clean_user}")
```

Test 1: Input Admin, Output admin

Test 2: Input USERNAME, Output username

Edge Case: Input (only spaces), Output is an empty string.

Explanation: strip() removes spaces from the ends, and lower() changes all letters to small letters.

Practice 2 – Name Formatter

Input: First: lAsYa, Last: kolluru

Expected Output: Lasya Kolluru

Code:

```
first = input("First Name: ")
last = input("Last Name: ")
full_name = f"{first.strip().title()} {last.strip().title()}"
print(f"Full Name: {full_name}")
```

Test 1: Input john and DOE, Output John Doe

Test 2: Input aLiCe and sMITH, Output Alice Smith

Edge Case: Input and smith, Output Smith

Explanation: strip() cleans the extra spaces, and title() capitalizes the first letter of each name. The f-string joins them into one sentence.

Practice 3 – Word Analyzer

Input: I am Lasya

Expected Output: Length: 10, First: I, Last: a, Spaces: 2, Vowels: 4

Code:

```
text = input("Sentence: ")

text_lower = text.lower()

vowels = text_lower.count('a') + text_lower.count('e') + text_lower.count('i') +
text_lower.count('o') + text_lower.count('u')

print(f"Length: {len(text)}")

print(f"First: {text[0]}")

print(f>Last: {text[-1]}")

print(f"Spaces: {text.count(' ')}")

print(f"Vowels: {vowels}")
```

Test 1: Input Hello World, Output: Length 11, First H, Last d, Spaces 1, Vowels 3

Test 2: Input Python, Output: Length 6, First P, Last n, Spaces 0, Vowels 1

Edge Case: Input a, Output: Length 1, First a, Last a, Spaces 0, Vowels 1

Explanation: len() counts total characters, [0] gets the first letter, and [-1] gets the last letter. count() finds exact matches for spaces and vowels.

Practice 4 – Email Inspector

Input: test@gmail.com

Expected Output: Has @: True, Valid End: True, @ Position: 4

Code:

```
email = input("Email: ")

has_at = '@' in email

valid_end = email.endswith('.com') or email.endswith('.in')

at_pos = email.find('@')

print(f"Has @: {has_at}")

print(f"Valid End: {valid_end}")

print(f"Position of @: {at_pos}")
```

Test 1: Input lasya@test.in, Output: True, True, 5

Test 2: Input lasycom, Output: False, True, -1

Edge Case: Input hello@, Output: True, False, 5

Explanation: in checks if the symbol exists. endswith() verifies the domain name. find() locates the exact index number of the symbol.

Practice 5 – Product Code Extractor

Input: IND-MOB-2026-001

Expected Output: Country: IND, Category: MOB, Year: 2026, Serial: 001

Code:

```
code = input("Code: ")
print(f"Country: {code[0:3]}")
print(f"Category: {code[4:7]}")
print(f"Year: {code}")
print(f"Serial: {code[13:]}")
```

Test 1: Input IND-MOB-2026-001, Output extracts IND, MOB, 2026, 001 correctly.

Test 2: Input USA-TAB-2025-002, Output extracts USA, TAB, 2025, 002 correctly.

Edge Case: Input UK-PC-26-01 (wrong format), Output slices the wrong characters.

Explanation: Slicing [start:end] cuts the long string into exact smaller pieces using their index numbers.

13. Part K – Debugging Lab

Bug 1

```
name = input('Name: ')
print(name[20])
```

Why can this fail? Fix it.

Corrected code / explanation: name = input("Name: ")

```
print(name[0])
```

name[20] tries to find the 21st letter of the name (since counting starts at 0). If the user types a short name, the program crashes. Fixing it to name[0] safely prints just the first letter.

Bug 2

```
age = '20'
print(f'Age: {age + 1}')
```

Fix it.

Corrected code / explanation: age = '20'

```
print(f'Age: {int(age) + 1}')
```

The '20' has quote marks around it, which makes it text (a string), not a number. Python cannot add text and a number (1) together. We use int() to turn the text into a real number first

Bug 3

```
name = 'phani'
age = 30
print('Hello ' + name + age)
```

Identify the type problem and fix it.

Corrected code / explanation: name = 'phani'

```
age = 30  
  
print('Hello ' + name + str(age))
```

age is a number, but name and 'Hello ' are text. You cannot glue text and numbers together using the + sign. We fix this type problem by turning the number into text using str(age).

Bug 4

```
email = input('Email: ')  
print(email.endswith('.com'))
```

Test with 3 inputs and explain.

Corrected code / explanation: email = input("Email: ")

```
print(email.endswith(".com"))
```

lasya@gmail.com -> True because it ends with .com

lasya@gmail.in -> False because it does not end with .com.

kolluru@gmail.com -> true because it ends with .com

Explanation: endswith(".com") checks whether the email ends with .com

Bug 5

```
code = 'IND-MOB-2026-001'  
print(code[20])
```

Find the valid index range and fix the requirement.

Corrected code / explanation: code = 'IND-MOB-2026-001'

```
print(code[0])
```

Explanation: The valid index range is 0 to 15. code[20] is invalid because the string has 16 characters.
give me telugu meaning in english words

Bug 6

```
text = ' hello world '  
print(text.upper().strip())
```

Does it work? Explain the order of operations.

Corrected code / explanation: text = ' hello world '

```
print(text.upper().strip())
```

Explanation: First, upper() changes the text to uppercase. Then, strip() removes the spaces from the beginning and end.

14. Part L – Main Project: Text-Cleaning & Profile Formatter

Build a practical Python program that accepts messy profile information and converts it into a clean, readable profile.

Scenario

A company receives profile data in inconsistent formats. Names may contain extra spaces, emails may use mixed cases, and cities may be entered randomly. Your program should clean the data and generate a professional profile summary.

Required Inputs

- Full name
- Email address
- Phone number
- City
- Job role
- Short bio

Must-Have Requirements

- Remove leading/trailing spaces.
- Format the full name using title().
- Convert email to lowercase.
- Format city and job role neatly.
- Calculate bio character count.
- Check whether the email contains '@'.
- Check whether the email ends with a supported domain such as .com or .in.
- Extract first name from the full name.
- Extract last name using string logic.
- Display the profile using f-strings.
- Show a small data-quality section.
- Test with messy uppercase/lowercase and extra spaces.

Expected Output Example

```
===== PROFILE =====
Name       : Phani Gorantla
First Name : Phani
Email      : phani@example.com
Phone     : 9876543210
City      : Hyderabad
Role      : Software Developer
Bio Length : 42 characters

----- DATA QUALITY -----
Email Valid : Yes
=====
```

Do not hard-code the example. The program must work with any user's input.

Code:

```
print("Enter Profile Details")

full_name = input("Full Name: ").strip().title()

email = input("Email: ").strip().lower()

phone = input("Phone Number: ").strip()

city = input("City: ").strip().title()

job_role = input("Job Role: ").strip().title()

bio = input("Short Bio: ").strip()

bio_length = len(bio)

email_has_at = "@" in email

email_has_domain = email.endswith(".com") or email.endswith(".in")

email_valid = {True: "Yes", False: "No"}[email_has_at and email_has_domain]

space_index = full_name.find(' ')

first_name = full_name[:space_index]

last_name = full_name[space_index + 1:]

print("\n===== PROFILE =====")

print(f"Name : {full_name}")

print(f"First Name : {first_name}")

print(f>Last Name : {last_name}")

print(f>Email : {email}")

print(f"Phone : {phone}")

print(f"City : {city}")

print(f"Role : {job_role}")

print(f"Bio Length : {bio_length} characters")

print("\n----- DATA QUALITY -----")

print(f>Email Valid : {email_valid}")

print("=====")
```

OutPut:

```
===== PROFILE =====
Name       : Lasya Kolluru
First Name : Lasya
Email      : lasya.kolluru@gmail.com
Phone     : 9876543210
City      : Nellore
Role      : Student
Bio Length : 44 characters

----- DATA QUALITY -----
Email Valid : Yes
=====
```

15. Project Bonus Challenges

- Clean repeated spaces inside a name.

```
raw_name = input("Full Name: ")

clean_name = " ".join(raw_name.split()).title()

print(f"Cleaned Name: {clean_name}")
```

output: Full Name: lasya kolluru
Cleaned Name: Lasya Kolluru

- Validate phone number length.

```
phone = input("Phone Number: ").strip()

phone_valid = len(phone) == 10

print(f"Phone Valid: {phone_valid}")
```

output: Phone Number: 9014975515
Phone Valid: True

- Check that the full name contains at least two words.

```
full_name = input("Full Name: ").strip()

has_two_words = " " in full_name

print(f"Has at least two words: {has_two_words}")
```

output: Full Name: kollurulasya
Has at least two words: False

- Generate initials from the name.

```
full_name = input("Full Name: ").strip().title()

space_index = full_name.find(' ')

first_name = full_name[:space_index]

last_name = full_name[space_index + 1:]

initials = first_name[0] + last_name[0]

print(f"Initials: {initials}")
```

output: Full Name: Lasya Kolluru
Initials: LK

- Generate a username from the person's name.

```
full_name = input("Full Name: ").strip()

space_index = full_name.find(' ')
```

```

first_name = full_name[:space_index].lower()

last_name = full_name[space_index + 1:].lower()

username = first_name + "_" + last_name

print(f"Username: {username}")

```

output: Full Name: Lasya Kolluru
Username: lasya_kolluru

- Create a one-line professional bio using an f-string.

```

full_name = input("Full Name: ").strip().title()

job_role = input("Job Role: ").strip().title()

city = input("City: ").strip().title()

pro_bio = f"{full_name} is a {job_role} based in {city}."

print(pro_bio)

```

output: Full Name: Lasya Kolluru
Job Role: Student
City: Nellore
Lasya Kolluru is a Student based in Nellore.

- Display a simple data-quality percentage.

```

email = input("Email: ").strip()

phone = input("Phone: ").strip()

full_name = input("Full Name: ").strip()

score = (

    ("@" in email and (email.endswith(".com") or email.endswith(".in")))) +

    (len(phone) == 10) +

    (" " in full_name)

)

quality_percentage = (score / 3) * 100

print(f>Data Quality Score:{quality_percentage}%")

```

output: Email: lasya.kolluru@gmail.com
Phone: 9876543210
Full Name: Lasya Kolluru
Data Quality Score: 100.0%

16. Project Test Cases

Test	Input condition	Expected behaviour	Actual result
1	Normal clean input	Clean profile	Clean profile will be displayed
2	Extra spaces around fields	Spaces removed	Extra spaces will be removed
3	Random uppercase/lowercase	Proper formatting	Proper formatting will be applied

4	Invalid-looking email	Flag data quality	Data quality will be flagged as No
5	Short/missing optional data	Program should handle it sensibly	Program will handle it without crashing

17. Explain Your Project

1. What is the difference between cleaning data and formatting data?

Cleaning removes unwanted things like extra spaces. Formatting changes how the text looks, like making the first letters capital.

2. Which string methods did you use and why?

I used `.strip()` to remove extra spaces, `.lower()` to make emails small letters, and `.title()` to make names start with capital letters.

3. Where did you use indexing?

I used indexing like `name[0]` to grab the very first letter to make initials.

4. Where did you use slicing?

I used slicing like `name[:space_pos]` to easily cut the full name into two parts: first name and last name.

5. Why are f-strings useful here?

They make it very easy to mix variables and text together in one sentence without using the `+` sign over and over.

6. What error did you face?

I tried to cut the name using a fixed number like `name[0:5]`, but it gave wrong answers for longer names. I fixed it by finding exactly where the space character was first.

7. How did you test edge cases?

I tested the program by typing bad inputs on purpose, like names with lots of extra spaces, emails without the `@` symbol, and messy capital letters.

8. If this processed 1 lakh profiles, what would you improve first?

I would add a loop so the program runs automatically for all profiles at once without me having to type each one by hand.

18. Final Self-Check

Skill	Yes	Need Practice	Notes
Use arithmetic/comparison/logical operators	☒	☐	I can use operators in Python
Explain <code>=</code> vs <code>==</code>	☒	☐	<code>=</code> assigns a value, <code>==</code> compares a value
Manipulate strings	☒	☐	I can work with

			strings
Use positive and negative indexing	☒	☐	I can access characters using indexes
Use slicing	☐	☒	I can access characters using indexes
Use string methods	☒	☐	I can use methods like upper(), lower(), strip().
Use in/not in	☒	☐	I can check whether a value is present
Use f-strings	☒	☐	I can insert variables into strings.
Debug IndexError/type problems	☒	☐	I can identify and fix common errors
Build the final text utility	☐	☒	I need more practice

19. Submission Checklist

- Operator experiments
- Operator questions
- String exploration
- String-method table
- Indexing/slicing challenge
- f-string exercises
- 30 MCQ answers
- 5 practice programs
- Debugging Lab
- Text-Cleaning & Profile Formatter
- Project test cases
- Project explanation and self-check